

Labwork 4: Threads

Le Quoc Trung
Student ID: 22BI13xxx

October 1, 2026

1 Goal

Take the grayscale code of labwork 3 and change the kernel launch from a 1D grid to 2D blocks, then measure the speedup for different 2D block sizes and compare it with the 1D version. Everything was run on Google Colab with a Tesla T4, on the same image as in labwork 3.

2 How I improved the labwork

In labwork 3 the image was flattened into a 1D array of (`pixelCount`, 3) and each thread found its pixel with a single index. Here the image keeps its original shape (`height`, `width`, 3) and every thread computes two coordinates, one from the x dimension and one from the y dimension of the block and the grid. The grid size becomes a pair instead of a single number, and each of its two components is the image size in that direction divided by the block size, rounded up.

The test at the beginning of the kernel is now on both coordinates, because neither 553 nor 491 is a multiple of the block size, so the blocks on the right edge and on the bottom edge contain threads with no pixel to work on.

One detail that matters: `threadIdx.x` is the fastest changing index, so I let x be the column and y be the row. Written the other way around the image still comes out correct, but the threads of one warp would read from different rows of the image, which breaks the coalescing of the memory access.

The computation itself is unchanged, and the output is identical to the CPU version and to the 1D version.

3 Results

The CPU version takes 0.2860 s. The 1D kernel, with a block size of 64, takes 0.000084 s, which is a speedup of 3424 times. Each GPU value below is the average of 20 launches after one warm-up launch. Everything in this report, including the 1D version, was measured in the same session so that the comparison is fair.

Block size	Threads	Time (ms)	Speedup
1×1	1	2.116	135
2×2	4	0.539	530
4×4	16	0.141	2028
8×8	64	0.075	3807
16×16	256	0.076	3776
32×32	1024	0.084	3387
32×1	32	0.077	3715
32×2	64	0.077	3719
32×4	128	0.077	3712
32×8	256	0.077	3704
32×16	512	0.079	3627
8×32	256	0.075	3810
16×8	128	0.075	3802
64×16	1024	0.083	3430

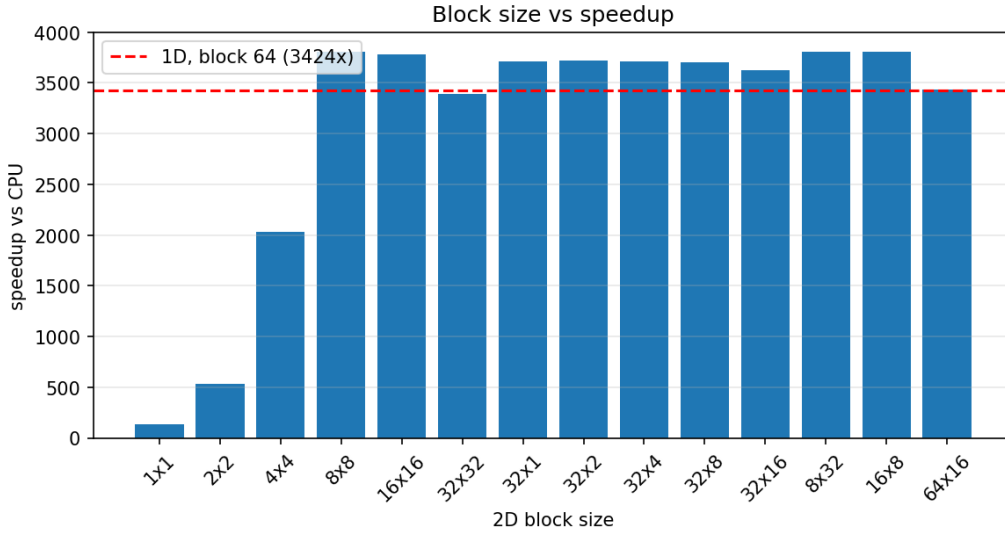


Figure 1: 2D block size vs speedup, with the 1D result as the dashed line

4 Exercises

Exercise 1. With 512 threads/block, 1024 threads/SM, 8 blocks/SM and 32 threads/warp, the best configuration is 16×16 . A 32×32 block would need 1024 threads, which is above the 512 limit per block, so it is not even allowed. A 8×8 block has 64 threads, and even with the maximum of 8 blocks per SM that gives only 512 threads out of 1024, so half of the SM capacity is wasted. A 16×16 block has 256 threads, and 4 of them fill the SM to its full 1024 threads while staying under the 8 block limit.

Exercise 2. With a maximum of 1536 threads and 4 blocks per SM, the best choice is 512 threads/block. With 128 threads the 4 blocks give only 512 threads, and with 256 they give 1024. With 512 threads, 3 blocks already reach 1536 and fill the SM. With 1024 threads only one block fits, since two of them would need 2048, so the SM holds 1024 threads.

Exercise 3. A vector of 2000 elements with 512 threads per block needs $\lceil 2000/512 \rceil = 4$ blocks, so the grid contains $4 \times 512 = 2048$ threads. The last 48 threads have no element to compute, which is why the kernel needs a test on the index before writing.